



Universidad de Ingeniería y Tecnología (UTEC)

CS6002 - Programación I

Proyecto: Juego de damas chinas

Informe del Proyecto - Laboratorio 16

Profesor: Henry Giovanni Gallegos Velgara

Integrantes:

Colonio Soto, Mayra Valeska - 20261PRECS01168

Mayhua Cabrera, Carlos Emilio - 20261PRECS01126

Salazar Lezcano, Adrián Gabriel Alejandro - 2061PRECS01097

RESUMEN

El presente informe se centrará en demostrar la capacidad de Python para ser el motor principal de un juego tradicional (damas chinas), aplicando unas reglas específicas para hacerlo una buena práctica de los temas vistos durante el desarrollo del curso. Valiéndose únicamente de los temas enseñados, se logró hacer un código capaz de filtrar, elegir y jugar. Finalmente, se analizó todo lo desarrollado durante el proyecto y el informe, con una sección de conclusiones y recomendaciones a tomar en cuenta, para el desarrollo de un próximo proyecto.

INTRODUCCIÓN

Las damas chinas son un juego de mesa que se juega hace más de cien años y que se volvió popular porque las reglas son simples, pero las partidas igual exigen pensar bien las jugadas. A diferencia de las damas tradicionales, acá no se capturan fichas: lo único que importa es ser el primero en cruzar todas las piezas hacia el lado contrario del tablero. Esto lo hace un buen ejercicio para practicar lógica de programación, porque hay que controlar coordenadas, turnos, movimientos válidos y saltos múltiples sin que el programa se caiga ante una entrada rara del usuario.

Para este proyecto se armó una versión de consola del juego, usando Python sin clases ni librerías gráficas, apoyándose solo en funciones, listas y diccionarios. La idea principal era reforzar el manejo de matrices (el tablero como lista de listas), validaciones de entrada y separación del código en módulos, además de simular un oponente automático que elige sus jugadas siguiendo una lógica simple de avance hacia la meta.

Existen varias versiones digitales de damas chinas, desde apps móviles hasta implementaciones para practicar algoritmos de búsqueda (como minimax para IA más avanzada). En este proyecto se optó por una inteligencia artificial sencilla, basada en reglas, que prioriza los saltos largos y evita mover fichas que ya llegaron a la meta, dejando como posible mejora futura una IA que evalúe el tablero de forma más completa.

DEFINICIONES

1. **Tablero:** Matriz de 8x8 representada como una lista de listas en Python. Cada casilla guarda un carácter: "X" para las fichas del jugador 1, "O" para las del jugador 2 o también la computadora y "_" para una casilla vacía.
2. **Coordenadas:** Es una forma de identificar una casilla combinando una letra de columna (A a H) y un número de fila (1 a 8), como por ejemplo "A8".
3. **Jugada:** movimiento que se ingresa con cuatro caracteres, dos para la casilla de origen y dos para la de destino (por ejemplo, "A8C6"), que el programa traduce a coordenadas internas (fila, columna).
4. **Movimiento:** Desplazamiento de una ficha a una casilla en diagonal hacia una casilla vacía.
5. **Salto:** Una ficha pasa por encima de otra para caer en una casilla vacía más adelante. El programa permite saltos múltiples en línea diagonal, validando que el camino esté libre de obstáculos para que el salto sea legal.
6. **Turno:** Indica a qué jugador le toca mover. Se guarda en una variable que va alternando entre "X" y "O" después de cada jugada válida.
7. **Computadora:** En el modo "Jugar vs. Computadora", elige sus movimientos siguiendo una función que prioriza los saltos largos y evita mover fichas que ya cruzaron al lado contrario.
8. **Partida:** registro de un juego ya jugado, como los jugadores, fecha, resultado y tablero final, que se guarda en el archivo partidas.json.

FUNCIONAMIENTO

El proyecto está separado en dos módulos: proyecto2.py, que solo contiene el menú principal y llama a las funciones según la opción elegida, y funciones.py, donde está toda la lógica del juego (creación del tablero, validación de jugadas, turnos, manejo de la computadora y guardado de partidas). Esta separación se hizo para que el archivo principal sea corto y fácil de leer, dejando el detalle técnico en el segundo módulo.

Menú principal

El archivo proyecto2.py muestra las cuatro opciones del menú en un bucle que se repite hasta que el usuario elige salir. Si se ingresa una opción que no es 1, 2, 3 o 0, simplemente se pide de nuevo sin que el programa se rompa:

```
from funciones import crear_tablero, registro, turno_computadora, turno_local

def main():
    while True:
        print("\nJUEGO DAMAS CHINAS")
        print("1. Comenzar Juego")
        print("2. Jugar vs Computadora")
        print("3. Juegos realizados")
        print("0. Salir")

        opcion = input("Opcion: ")

        if opcion == "1":
            tablero = crear_tablero()
            turno_local(tablero)
        elif opcion == "2":
            tablero = crear_tablero()
            turno_computadora(tablero)
        elif opcion == "3":
            registro()
        elif opcion == "0":
            break
        else:
            print("Opcion invalida, vuelva a intentarlo")
```

Creación del tablero

La función crear_tablero arma la matriz de 8x8 recorriendo cada fila y columna. Las dos primeras filas se llenan con fichas "O" en las casillas de un color (usando la suma fila + columna para alternar), las dos últimas con fichas "X", y el resto del tablero queda vacío:

```
def crear_tablero():
    tablero = []
    for fil in range(8):
        filasT = []
        for colum in range(8):
            suma = fil + colum
            if fil in [0, 1]:
                filasT.append("O" if suma % 2 == 1 else "_")
            elif fil in [6, 7]:
                filasT.append("X" if suma % 2 == 1 else "_")
            else:
                filasT.append("_")
        tablero.append(filasT)
    return tablero
```

Traducción de coordenadas

Como el usuario escribe sus jugadas como texto (por ejemplo, "A8C6"), la función traductor se encarga de convertir esas letras y números en coordenadas que el programa pueda usar internamente (fila, columna). De paso, revisa que el formato sea válido y que el jugador no haya puesto algo fuera del tablero; si todo está mal, devuelve False, y si el jugador escribe "-1", se interpreta como cancelar la partida.

```
def traductor(jugada):
    if jugada == str(-1):
        return "Cancelado"
    if len(jugada) != 4:
        return False

    Columnas = {"A": 0, "B": 1, "C": 2, "D": 3, "E": 4, "F": 5, "G": 6, "H": 7}
    letra_original = jugada[0].upper()
    if not jugada[1].isdigit():
        return False
    numero_original = int(jugada[1]) - 1

    letra_final = jugada[2].upper()
    if not jugada[3].isdigit():
        return False
    numero_final = int(jugada[3]) - 1

    if letra_original not in Columnas or letra_final not in Columnas:
        return False
    if not (0 <= numero_original <= 7 and 0 <= numero_final <= 7):
        return False

    return (numero_original, Columnas[letra_original]), (numero_final, Columnas[letra_final])
```

Validación de movimientos

La función verificador es el corazón de las reglas del juego. Revisa que la ficha que se quiere mover sea del jugador que tiene el turno, que la casilla de destino esté vacía y que el movimiento sea en diagonal con una distancia permitida (1 para movimiento simple, o 2, 4 y 6 para saltos). Cuando la distancia es mayor a 1, llama a validar_camino_salto para confirmar que en el camino haya fichas para saltar y que las casillas intermedias de aterrizaje estén libres:

```
def verificador(tuplaoriginal, tuplafinal, tablero_actual, turno):
    posibles = [1, 2, 4, 6]
    filaOR, columnaOR = tupleoriginal
    filaFI, columnaFI = tuplafinal

    if tablero_actual[filaOR][columnaOR] != turno:
        return False
    if tablero_actual[filaFI][columnaFI] != "_":
        return False

    distancia_fil = abs(filaFI - filaOR)
    distancia_col = abs(columnaFI - columnaOR)
    if distancia_col != distancia_fil:
        return False
    if distancia_fil not in posibles:
        return False
    if distancia_fil == 1:
        return True
    return validar_camino_salto(tablero_actual, tupleoriginal, tuplafinal, distancia_fil)
```

Turnos y partidas entre dos jugadores

La función `turno_local` maneja todo el ciclo de una partida entre dos personas: pide y confirma los nombres, registra la fecha y hora con `datetime`, muestra el tablero en cada turno y pide el movimiento al jugador correspondiente. En cada vuelta revisa si ya hay un ganador (con la función `hay_ganador`, que cuenta cuántas fichas de cada jugador llegaron al lado contrario) o si el jugador en turno se quedó sin movimientos posibles (rey ahogado, que termina la partida en empate). Al terminar la partida, sea por victoria, empate o cancelación, se llama a `guardar_partida` para dejar el registro en el archivo JSON.

```
def turno_local(tablero):
    global ULTIMO_TABLERO
    print("Jugador 1, comienza la partida")
    Jugador_1, Jugador_2 = pedir_nombre_jugador()
    fichas = {Jugador_1: "X", Jugador_2: "O"}
    Jugador = Jugador_1
    turno_actual = fichas[Jugador]
    fecha = datetime.now().strftime("%d/%m/%y %H:%M")
    print(f"Juego {Jugador_1} vs {Jugador_2} {fecha}")
    print("↵1: Cancelar juego")

    while True:
        if hay_ganador(tablero) == "X":
            print(f"Ganador: {Jugador_1}")
            ULTIMO_TABLERO = [fila.copy() for fila in tablero]
            guardar_partida(Jugador_1, Jugador_2, fecha, Jugador_1)
            break
        if hay_ganador(tablero) == "O":
            print(f"Ganador: {Jugador_2}")
            ULTIMO_TABLERO = [fila.copy() for fila in tablero]
            guardar_partida(Jugador_1, Jugador_2, fecha, Jugador_2)
            break

        mostrar_tablero(tablero)
        if not posibles_jugadas(tablero, turno_actual):
            print("partida terminada por rey ahogado")
            ULTIMO_TABLERO = [fila.copy() for fila in tablero]
            guardar_partida(Jugador_1, Jugador_2, fecha, "Empate")
            break

        movimiento_realizado = input(f"{Jugador} Movimiento(formato A8C6): ")
        coordenadas = traductor(movimiento_realizado)
        if coordenadas == "Cancelado":
            print("Juego cancelado")
            ULTIMO_TABLERO = [fila.copy() for fila in tablero]
            guardar_partida(Jugador_1, Jugador_2, fecha, "Cancelado")
            break
        if coordenadas is False:
            print("Jugada inválida, formato A8C6")
```

Modo contra la computadora

turno_computadora sigue una lógica parecida, pero antes de pedir el movimiento revisa de quién es el turno: si le toca a la computadora, llama a `elegir_jugada_computadora`, que arma la lista de jugadas posibles para esa ficha (con `posibles_jugadas`), descarta las fichas que ya llegaron a la meta cuando hay otra opción disponible, y entre las jugadas que quedan prioriza los saltos largos hacia adelante; si no hay saltos disponibles, elige cualquier jugada válida al azar. De esta forma, la computadora siempre hace un movimiento legal y trata de avanzar hacia la victoria en vez de moverse al azar sin sentido.

```
def elegir_jugada_computadora(tablero, ficha):
    jugadas = posibles_jugadas(tablero, ficha)
    meta = [6, 7]
    pendientes = []
    for valor in jugadas:
        (Fil_or, _), _ = valor
        if Fil_or not in meta:
            pendientes.append(valor)
    if pendientes:
        jugadas = pendientes

    multiples = []
    for valor in jugadas:
        (Fil_or, _), (Fil_fi, _) = valor
        distancia_fil = abs(Fil_fi - Fil_or)
        if distancia_fil in [4, 6] and Fil_fi > Fil_or:
            multiples.append(valor)
    if multiples:
        return random.choice(multiples)
    return random.choice(jugadas)
```

Registro y guardado de partidas

Cada vez que termina una partida (gane alguien, haya empate o se cancele), `guardar_partida` arma un diccionario con los jugadores, la fecha, el resultado y el tablero final, y lo agrega a la lista que se guarda en `partidas.json`. La opción 3 del menú llama a `registro`, que lee ese archivo, muestra una tabla con todas las partidas jugadas y permite elegir una para ver cómo quedó el tablero al final de esa partida en particular.

```
def guardar_partida(jugador1, jugador2, fecha, resultado):
    """Guarda un juego en el archivo local de partidas."""
    global ULTIMO_TABLERO
    partidas = cargar_partidas()
    partida = {
        "jugador1": jugador1,
        "jugador2": jugador2,
        "fecha": fecha,
        "resultado": resultado,
        "tablero": ULTIMO_TABLERO if ULTIMO_TABLERO is not None else [],
    }
    partidas.append(partida)
    with open(PARTIDAS_PATH, "w", encoding="utf-8") as handle:
        json.dump(partidas, handle, indent=2)
```

EJEMPLOS DE EJECUCIÓN

A continuación se muestra una ejecución real del programa, jugando una partida local entre la computadora y el jugador "mai" hasta cancelarla con la opción -1. Se puede ver el menú, el registro del nombre, el tablero inicial y cómo se actualiza después de cada jugada:

```
JUEGO DAMAS CHINAS
1. Comenzar Juego
2. Jugar vs Computadora
3. Juegos realizados
0. Salir
Opcion: 2
El futuro está en tus manos
Nombre del jugador : mai
Confirmar nombre (S/N): s
Juega primero? (S/N): n
Juego mai vs Computador (Alias Mechanus Calsen) 03/07/26 02:39
ã'1: Cancelar juego

  A B C D E F G H      A B C D E F G H
1 _ 0 _ 0 _ 0 _ 0 1    1 _ 0 _ 0 _ 0 _ 0 1
2 0 _ 0 _ 0 _ 0 _ 2    2 0 _ 0 _ 0 _ 0 _ 2
3 _ _ _ _ _ _ _ 3      3 _ _ _ _ _ _ 0 3
4 _ _ _ _ _ _ _ 4      4 _ _ _ _ _ _ _ 4
5 _ _ _ _ _ _ _ 5      5 _ _ _ _ _ _ _ 5
6 _ _ _ _ _ _ _ 6      6 _ _ _ _ _ _ _ 6
7 _ X _ X _ X _ X 7    7 _ X _ X _ X _ X 7
8 X _ X _ X _ X _ 8    8 X _ X _ X _ X _ 8
  A B C D E F G H      A B C D E F G H
Computador ? G2H3      mai ?(formato A8C6): E8F7
  A B C D E F G H      A B C D E F G H
1 _ 0 _ 0 _ 0 _ 0 1    1 _ 0 _ 0 _ 0 _ 0 1
2 0 _ 0 _ 0 _ 0 _ 2    2 0 _ 0 _ 0 _ 0 _ 2
3 _ _ _ _ _ _ _ 3      3 _ _ _ _ _ _ 0 3
4 _ _ _ _ _ _ _ 4      4 _ _ _ _ _ _ _ 4
5 _ _ _ _ _ _ _ 5      5 _ _ _ _ _ _ _ 5
6 _ _ _ _ _ _ _ 6      6 _ _ _ _ _ _ _ 6
7 _ X _ X _ X _ X 7    7 _ X _ X _ X _ X 7
8 X _ X _ X _ X _ 8    8 X _ X _ X _ X _ 8
  A B C D E F G H      A B C D E F G H
mai ?(formato A8C6): H7G6      Computador ? E2F3
  A B C D E F G H      A B C D E F G H
1 _ 0 _ 0 _ 0 _ 0 1    1 _ 0 _ 0 _ 0 _ 0 1
2 0 _ 0 _ 0 _ 0 _ 2    2 0 _ 0 _ 0 _ 0 _ 2
3 _ _ _ _ _ _ _ 3      3 _ _ _ _ _ _ 0 3
4 _ _ _ _ _ _ _ 4      4 _ _ _ _ _ _ _ 4
5 _ _ _ _ _ _ _ 5      5 _ _ _ _ _ _ _ 5
6 _ _ _ _ _ _ _ 6      6 _ _ _ _ _ _ _ 6
7 _ X _ X _ X _ X 7    7 _ X _ X _ X _ X 7
8 X _ X _ X _ X _ 8    8 X _ X _ X _ X _ 8
  A B C D E F G H      A B C D E F G H
Computador ? E2F3      mai ?(formato A8C6): F7E6
  A B C D E F G H      A B C D E F G H
1 _ 0 _ 0 _ 0 _ 0 1    1 _ 0 _ 0 _ 0 _ 0 1
2 0 _ 0 _ 0 _ 0 _ 2    2 0 _ 0 _ 0 _ 0 _ 2
3 _ _ _ _ _ _ _ 3      3 _ _ _ _ _ _ 0 3
4 _ _ _ _ _ _ _ 4      4 _ _ _ _ _ _ _ 4
5 _ _ _ _ _ _ _ 5      5 _ _ _ _ _ _ _ 5
6 _ _ _ _ _ _ _ 6      6 _ _ _ _ _ _ _ 6
7 _ X _ X _ X _ X 7    7 _ X _ X _ X _ X 7
8 X _ X _ X _ X _ 8    8 X _ X _ X _ X _ 8
  A B C D E F G H      A B C D E F G H
mai ?(formato A8C6): F7E6      Computador ? F3E2
  A B C D E F G H      A B C D E F G H
1 _ 0 _ 0 _ 0 _ 0 1    1 _ 0 _ 0 _ 0 _ 0 1
2 0 _ 0 _ 0 _ 0 _ 2    2 0 _ 0 _ 0 _ 0 _ 2
3 _ _ _ _ _ _ _ 3      3 _ _ _ _ _ _ 0 3
4 _ _ _ _ _ _ _ 4      4 _ _ _ _ _ _ _ 4
5 _ _ _ _ _ _ _ 5      5 _ _ _ _ _ _ _ 5
6 _ _ _ _ _ _ _ 6      6 _ _ _ _ _ _ _ 6
7 _ X _ X _ X _ X 7    7 _ X _ X _ X _ X 7
8 X _ X _ X _ X _ 8    8 X _ X _ X _ X _ 8
  A B C D E F G H      A B C D E F G H
Computador ? F3E2      mai ?(formato A8C6): -1
Juego cancelado

JUEGO DAMAS CHINAS
1. Comenzar Juego
2. Jugar vs Computadora
3. Juegos realizados
0. Salir
Opcion: 0
```

Como se ve en la salida, la partida comienza con la computadora moviendo su ficha O de G2 a H3, un movimiento simple en diagonal. Luego el jugador "mai" responde moviendo de H7 a G6, avanzando su ficha X hacia el lado contrario. La partida continúa con varios intercambios: la computadora mueve E2 a F3, "mai" responde con F7 a E6, y así sucesivamente, ambos avanzando sus fichas en diagonal hacia la meta del rival. En el turno del computador se puede ver también cómo retrocede una ficha (F3 a E2) cuando le conviene reposicionarse. Hacia el final, "mai" tiene fichas en E6 y G6 ya bastante avanzadas, mientras la computadora tiene piezas en F3 y H3 acercándose al lado X. Al ingresar -1 en su turno, mai cancela la partida, la cual queda registrada con resultado "Cancelado" en el archivo partidas.json junto con el estado final del tablero.

CONCLUSIONES

- Dividir el código en dos archivos fue una buena decisión porque cuando algo fallaba, era más fácil encontrar dónde estaba el problema, en vez de buscar en un solo archivo gigante con todo mezclado.
- Usar listas de listas para el tablero funcionó bien y no necesitamos nada más complicado, lo que nos sorprendió porque al inicio pensábamos que iba a ser más difícil manejar los saltos múltiples con esa estructura.
- Gran parte del tiempo de desarrollo lo pasamos arreglando errores que venían de entradas raras del usuario, como nombres vacíos o jugadas mal escritas, así que aprendimos que validar las entradas desde el principio ahorra bastante dolor de cabeza después.
- La computadora no es un genio, pero cumple: siempre hace jugadas válidas y trata de avanzar, lo que hace que las partidas contra ella sean entretenidas, aunque no muy desafiantes.
- El trabajar con variables con nombres sencillos y expositivos, es sumamente importante, en proyectos de gran envergadura.

RECOMENDACIONES

- La IA debería poder anticipar nuestras jugadas en vez de solo pensar en avanzar sus propias fichas; así las partidas serían más parejas y el juego tendría más replay value.
- Una mejora interesante sería que la computadora reconozca cuando el jugador tiene una ficha a punto de llegar a la meta y la bloquee, en vez de ignorarlo completamente como hace ahora.
- Hubiera sido útil hacer pruebas automáticas para las funciones de validación desde el principio, porque cada vez que cambiábamos algo, teníamos que probar manualmente si todo seguía funcionando bien.
- Planear bien las funciones antes de ponerse a escribir código habría ahorrado tiempo, porque varias veces tuvimos que reorganizar cosas que ya estaban hechas porque no encajaban bien con el resto.
- Finalmente, el utilizar un tablero visualmente más grande pudo ser una buena decisión, ayudando a la capacidad de jugar de forma óptima.